

Strings vs Character Arrays

Character Arrays	String
<code>char str[50];</code>	<code>string str;</code>
<code>cin>>str;</code>	<code>cin>>str;</code>
<code>cin.get(str, 50);</code>	<code>getline(cin,str) ;</code>
<code>cin.getline(str , 50 , '\$')</code>	<code>getline(cin,str,'\$') ;</code>
<code>cout<<str;</code>	<code>cout<<str;</code>

1

Pointer and Reference

First-year Cybersecurity Students
Lecturer Rana AlShami

Pointer Variables

- Pointer variable: content is a memory address
- A pointer is a **special type of variable** designed to store the memory location of another variable that shares the **same data type**
- its support dynamic memory allocation routines.
- Declaring Pointer Variables: Syntax
- Examples:

```
int *p;      char *ch;
```

C++ Programming: From Problem Analysis to Program Design, Second Edition

3

Pointer Variables (continued)

- C++ does not automatically initialize variables.
- `p = NULL;`
- These statements are equivalent

```
int    *p;
int*   p;
int    * p;
```

- The character `*` can **appear anywhere** between type name and variable name

C++ Programming: From Problem Analysis to Program Design, Second Edition

4

Pointer Variables (continued)

- To avoid confusion, attach the character `*` to the variable name

```
int    *p, q;
```

only `p` is the pointer variable, not `q`; here `q` is an `int` variable

- The following statement declares both `p` and `q` to be pointer variables of the type `int`

```
int    *p, *q;
```

C++ Programming: From Problem Analysis to Program Design, Second Edition

5

Dereferencing Operator (*)

- C++ uses `*` as the `binary` multiplication operator and as a `unary` operator
- When used as a unary operator, `*`
 - Called `dereferencing operator` or `indirection operator`

C++ Programming: From Problem Analysis to Program Design, Second Edition

6

Address of Operator (&)

- The ampersand, &, is called **the address of operator**
- The address of operator is a **unary operator** that returns the **address of its operand**

C++ Programming: From Problem Analysis to Program Design, Second Edition

7

```
int x = 25;  
int *p;  
p = &x;    //store the address of x in p
```

The following statement prints the value stored in the memory space pointed to by `p`, which is the value of `x`.

```
cout << *p << endl;
```

The following statement stores 55 in the memory location pointed to by `p`—that is, in `x`.

```
*p = 55;
```

```
int *p;  
int num;
```

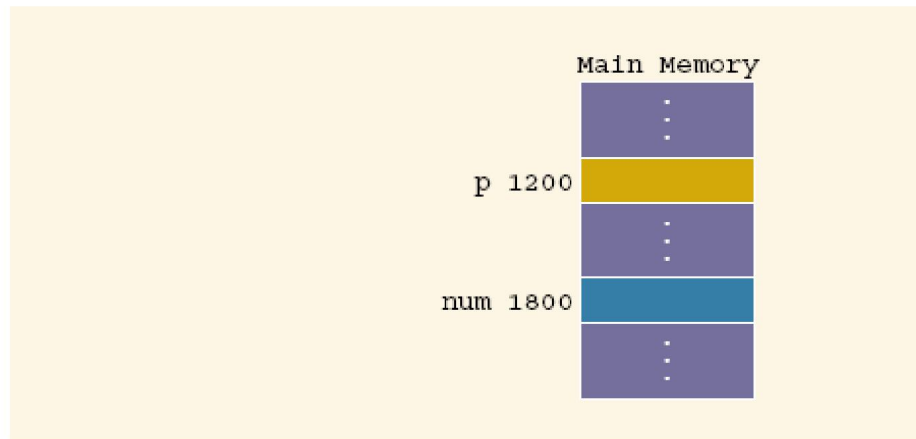


FIGURE 14-1 Main memory, p, and num

```
num = 78;
```

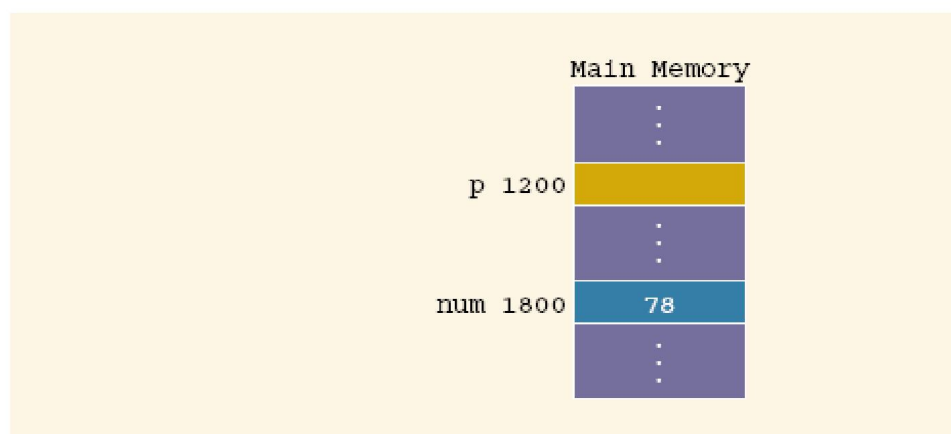


FIGURE 14-2 num after the statement num = 78; executes

```
p = &num;
```

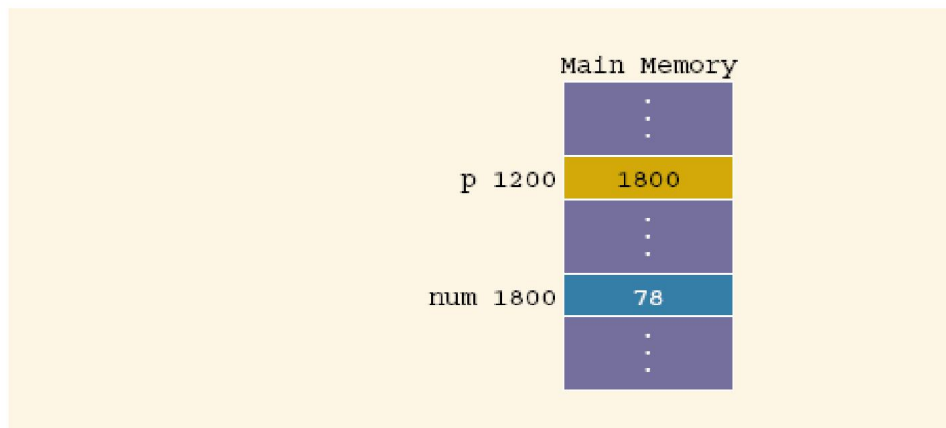


FIGURE 14-3 p after the statement `p = #` executes

```
*p = 24;
```

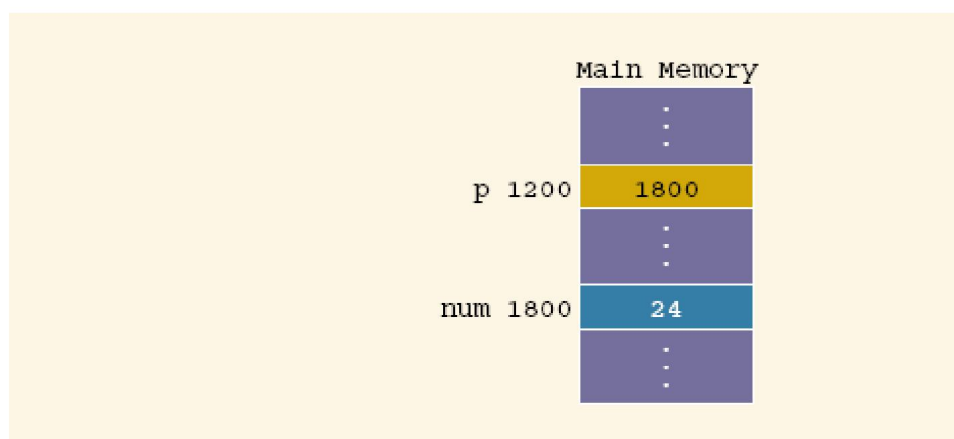
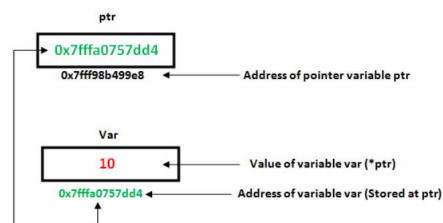


FIGURE 14-4 *p and num after the statement `*p = 24;` executes



Note

- The numbers 1200 and 1800 are used only to simplify the example and do not represent real memory addresses.
- Memory addresses are typically shown in **hexadecimal** form, even though internally they're just binary numbers.



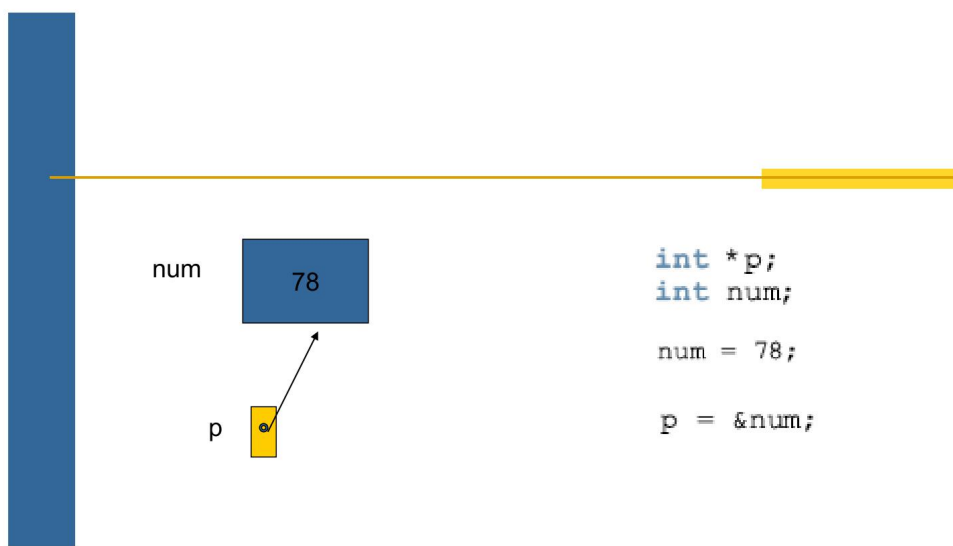
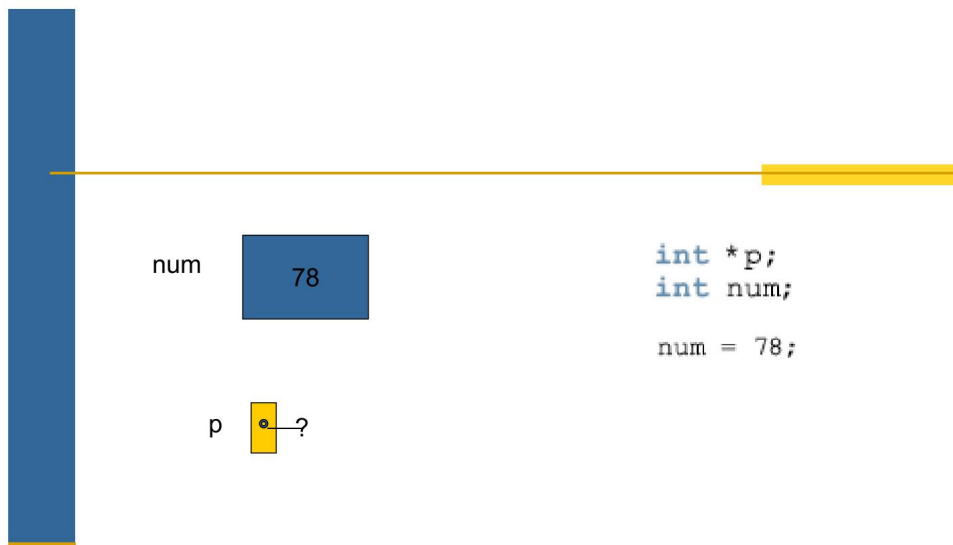
num

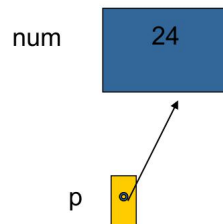


```
int *p;
int num;
```

p







```
int *p;
int num;

num = 78;

p = &num;

*p = 24;
```

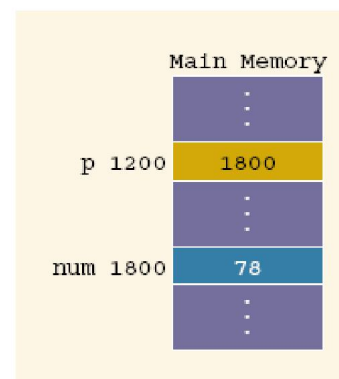


Note

Pointer variables:

$\&p$, p , and $*p$ all have different meanings.

1. $\&p$ means the address of p —that is, **1200**
2. p means the content of p (**1800**). (Address)
3. $*p$ means the content (78) (Value)



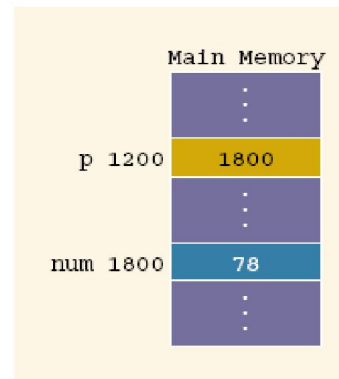
`int p = #` executes



Note

Normal variables:

- `int num;`
- `&num` means the address of `num` = 1800
- `num` mean the value of `x` = 78

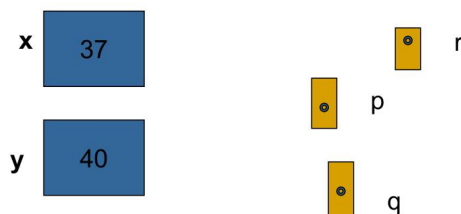


`int p = #` executes

19

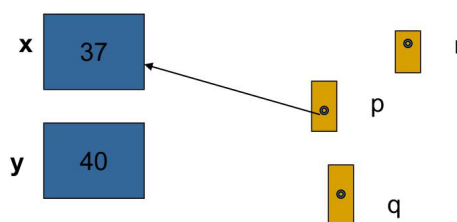
Chapter 14: An example with pointers

```
int *p;
int *q;
int *r;
int x = 37;
int y = 40;
```



An example with pointers

```
p = &x;  
q = &y;  
r = p;
```

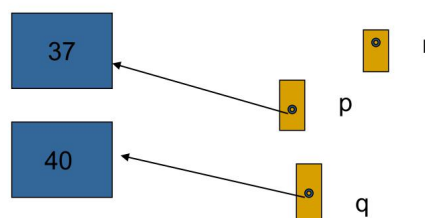


C++ Programming: From Problem Analysis to Program Design, Second Edition

21

An example with pointers

```
p = &x;  
q = &y;  
r = p;
```

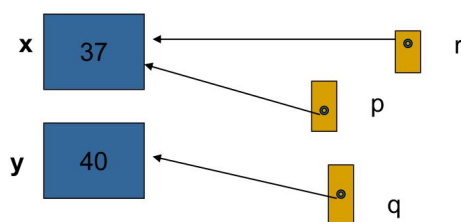


C++ Programming: From Problem Analysis to Program Design, Second Edition

22

An example with pointers

```
p = &x;
q = &y;
r = p;
```

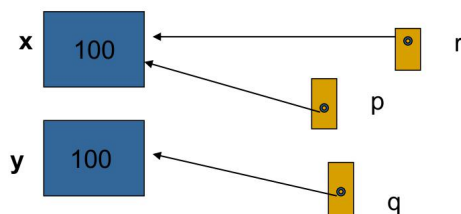


C++ Programming: From Problem Analysis to Program Design, Second Edition

23

An example with pointers

```
x = 100;
y = 100;
( p == q ) is false
( *p == *q ) is true
( p == r ) is true
```



Pointers to pointers

- `char a;`
- `char *b;`
- `char ** c;`
- `a = 'g';`
- `b = &a;`
- `c = &b;`
- The syntax simply requires the unary operator (*) for each level of indirection while declaring the pointer

25

Constant with Pointer

- Constant Pointer: The pointer's address can't change; it always points to the same place, but the value there can change.
- Pointer to a Constant: The pointer can point to different addresses, but the value at those addresses can't be changed.

26

Case	Pointer Definition	Can change value?	Can change address?	Example
1	<code>const int *p2 = &x;</code> Pointer to const int	No	Yes	<code>*p2 = 50; // error</code> <code>p2 = &y; // OK</code>
2	<code>int *const p3 = &x;</code> Const pointer to int	Yes	No	<code>*p3 = 60; // OK</code> <code>p3 = &y; // error</code>
3	<code>const int *const p4 = &x;</code> Const pointer to const int	No	No	<code>*p4 = 90; // error</code> <code>p4 = &y; // error</code>

27

Reference

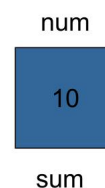
- A reference variable is an **alias** for an existing variable.

28

L-value Reference (&):

1. L-value Reference (&) (left-value)

- An l-value reference binds to an **existing variable** (l-value) that has a definite memory location. It allows **direct access and modification** of the original variable.
- `DataType &refName = existingVariable;`
- **Example:**
 - `int num = 10;`
 - `int &sum = num; // sum references num`
 - **Note:** sum and num share the same memory. Changes to sum affect num.



29

R-value Reference (&&)

2. An r-value reference binds to a temporary value (r-value). (Right value)

- Syntax:
- `DataType &&refName = Value;`
- **Example**
 - `int x=50`
 - `int &&rref = 5;`
 - `rref = 10;`
 - `int &&rref = x; // Error`

30

Full Example

- `int x=5;`
- `int &ref=x;`
- `int *p=&x;`
- `int **ptp=&p;`

name	x or ref	p	ptp
value	5	1000	2000
address	1000	2000	3000

Variable	Stored Value	What It Represents
x	5	The original integer value
ref	5	Reference (alias) to x — same memory as x
p	1000	Address of x
ptp	2000	Address of pointer p

31

Full Example

- `int x=5;`
- `int &ref=x;`
- `int *p=&x;`
- `int **ptp=&p;`

name	x or ref	p	ptp
value	5	1000	2000
address	1000	2000	3000

To print value of **x = 5**

1. `cout << "print x value by name " << x << endl;`
2. `cout << "print x value by ref " << ref << endl;`
3. `cout << "print p value " << *p << endl;`
`*p=*(1000)=5 *p--> x=5`
4. `cout << "print x value by pointer ptp" << ** ptp << endl;`
`**ptp= (*(2000))= *(1000)=5 **ptp --> *p---> x=5`

Full Example

- `int x=5;`
- `int &ref=x;`
- `int *p=&x;`
- `int **ptp=&p;`

name	x or ref	p	ptp
value	5	1000	2000
address	1000	2000	3000

To print address of `x = 1000`

1. `cout << "print x address by variables x " << &x << endl;`
2. `cout << "print x address by pointer p " << p << endl;`
3. `cout << "print x address by pointer ptp " << *ptp << endl;`
4. `cout << "print x address by ref " << &ref << endl;`

33

Full Example

- `int x=5;`
- `int &ref=x;`
- `int *p=&x;`
- `int **ptp=&p;`

name	x or ref	p	ptp
value	5	1000	2000
address	1000	2000	3000

`cout << "print p address " << &p << endl; =2000`

`cout << "print ptp address " << &ptp << endl; =3000`

34

```

#include <iostream>
using namespace std;
int main()
{
    int x=5;
    int &ref=x;
    int *p=&x;
    int ** ptp=&p;
    cout << "print x value by name " << x << endl;
    cout << "print p value " << *p << endl;
    cout << "print x value by pointer ptp " << ** ptp << endl;
    cout << "print x value by ref " << ref << endl;
    cout << "print x address by variables x " << &x << endl;
    cout << "print x address by pointer p " << p << endl;
    cout << "print x address by pointer ptp " << *ptp << endl;
    cout << "print x address by pointer ref " << &ref << endl;
    cout << "print p address " << &p << endl;
    cout << "print ptp address " << &ptp << endl;
    return 0;
}

```

35

```

print x value by name 5
print p value 5
print x value by pointer ptp 5
print x value by ref 5
print x address by variables x 0x7ffe4de9a124
print x address by pointer p 0x7ffe4de9a124
print x address by pointer ptp 0x7ffe4de9a124
print x address by pointer ref 0x7ffe4de9a124
print p address 0x7ffe4de9a118
print ptp address 0x7ffe4de9a110

```

Address in memory

36